

MINI-GRAMMAR SPECIFICATION

Compiler Construction Assignment 1

Group Number: 3

Group Members

1. KARUGA LEO MUNENE - SCS3/146815/2023
2. OCHANDA ERIC KUMENDA - SCS3/6959/2023
3. WANGARI MORRIS NGUMO - P15/1906/2022
4. KOECH SANDRA CHEROP - SCS3/146544/2023

Course: SCS3305 - Compiler Construction

Date: 12 March 2026

1. Introduction

This report presents the mini-grammar specification for a small but expressive fragment of a C-like programming language, designed as the foundation for a compiler construction project. The language, referred to as Elmos, is deliberately scoped to be implementable within a single semester while demonstrating all essential compiler construction concepts.

The mini-language includes the following constructs as a minimum:

- 5 token types: keywords, identifiers, integer literals, string literals and operators
- 2 control structures: if/else and while
- 2 expression types: arithmetic expressions and relational/logical expressions

The language is modeled after a subset of C, which provides a well-understood reference for grammar design and directly supports the implementation language chosen by the group (C with Flex and Bison).

2. Mini-Grammar Specification (BNF)

The following Backus-Naur Form (BNF) grammar defines the full syntax of Elmos. Non-terminals are enclosed in angle brackets < >, and terminals are written in bold or quoted form.

2.1 Program Structure

```

<program>      ::= <stmt_list>
<stmt_list>    ::= <statement> | <stmt_list> <statement>
<statement>    ::= <decl_stmt>
                | <assign_stmt>
                | <if_stmt>
                | <while_stmt>
                | <return_stmt>

```

2.2 Declaration and Assignment

```

<decl_stmt>    ::= 'int' IDENTIFIER ';'
                | 'int' IDENTIFIER '=' <expr> ';'
<assign_stmt> ::= IDENTIFIER '=' <expr> ';'

```

2.3 Control Structures

```

<if_stmt>      ::= 'if' '(' <cond_expr> ')' '{' <stmt_list> '}'
                | 'if' '(' <cond_expr> ')' '{' <stmt_list> '}'
                  'else' '{' <stmt_list> '}'

```

```
<while_stmt> ::= 'while' '(' <cond_expr> ')' '{' <stmt_list> '}'
```

2.4 Return Statement

```
<return_stmt> ::= 'return' <expr> ';' ;
```

2.5 Expressions

```
<expr> ::= <arith_expr>
        | STRING_LITERAL
```

```
<arith_expr> ::= <arith_expr> '+' <term>
                | <arith_expr> '-' <term>
                | <term>
```

```
<term> ::= <term> '*' <factor>
          | <term> '/' <factor>
          | <factor>
```

```
<factor> ::= '(' <arith_expr> ')'
           | IDENTIFIER
           | NUMBER
```

2.6 Conditional Expressions

```
<cond_expr> ::= <arith_expr> <relop> <arith_expr>
              | <cond_expr> '&&' <cond_expr>
              | <cond_expr> '||' <cond_expr>
              | '!' <cond_expr>
```

```
<relop> ::= '==' | '!=' | '<' | '>' | '<=' | '>='
```

3. Lexical Specification

This section provides the formal lexical specification for each token type in Elmos, including an English description, the corresponding regular expression, and notes on recognition priority.

3.1 Token Types

Token 1: Keywords

Description: Reserved words that have fixed meaning in Elmos and cannot be used as identifiers. The complete keyword set is: int, if, else, while, return.

Regular Expression:

```
keyword -> 'int' | 'if' | 'else' | 'while' | 'return'
```

Note: Keywords must be recognised before identifiers. A longest-match rule ensures that 'integer' is not tokenised as keyword 'int' followed by 'eger' — it is an identifier.

Token 2: Identifiers

Description: A sequence of letters (upper or lowercase), digits, and underscores, starting with a letter or underscore. Used to name variables.

Regular Expression:

```
identifier -> [a-zA-Z_][a-zA-Z0-9_]*
```

Token 3: Integer Literals (NUMBER)

Description: A non-empty sequence of decimal digits (0-9). No leading zeros (except the literal '0' itself). No floating-point or negative literals — negation is handled as a unary operator in the grammar.

Regular Expression:

```
number -> 0 | [1-9][0-9]*
```

Token 4: String Literals

Description: A sequence of any characters (except an unescaped double-quote or newline) enclosed in double quotes. Supports the escape sequences \n, \t, and \\..

Regular Expression:

```
string -> '"' ([^"\n\\] | '\\\' ['\\\' 'n' 't' '\"])* '"'
```

Token 5: Operators and Punctuation

Description: Single and double character tokens that act as operators, delimiters, or punctuation in Elmos.

Regular Expression:

```
operator -> '+' | '-' | '*' | '/'
          | '=' | '==' | '!=' | '<' | '>' | '<=' | '>='
          | '&&' | '||' | '!'
          | ';' | '(' | ')' | '{' | '}'
```

Whitespace and Comments (Ignored)

Description: Spaces, tabs, and newlines are not tokens — they are consumed and discarded. Single-line comments beginning with // are also discarded.

```
whitespace -> [ \t\n\r]+
comment    -> '//' [^\n]* '\n'
```

4. Token Summary Table

Token Type	Example Lexeme	Regular Expression
KEYWORD	int, if, while	'int' 'if' 'else' 'while' 'return'
IDENTIFIER	x, result, count	[a-zA-Z_] [a-zA-Z0-9_]*
NUMBER	0, 42, 100	0 [1-9] [0-9]*
STRING	"hello"	"([^\"] \\.)*"
OPERATOR	+, ==, <=, &&	see Section 3.1 Token 5
PUNCTUATION	; () { }	[;(){}]